Lec-17

10/06/26 7:04 PM

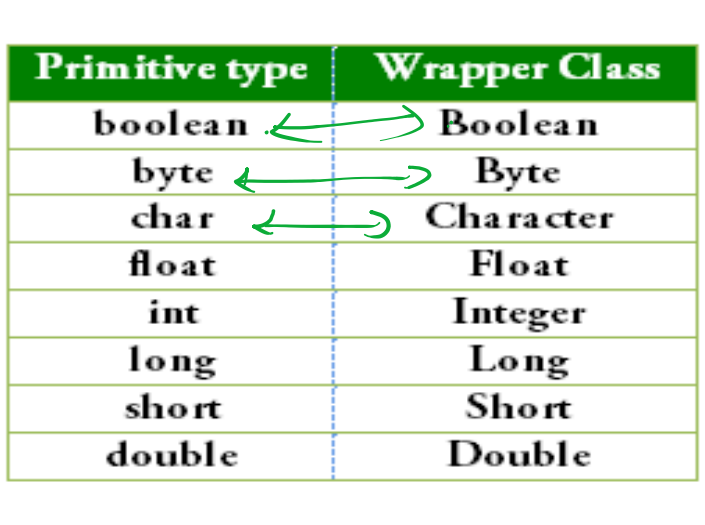
int ✗

Wrapper class

primitive

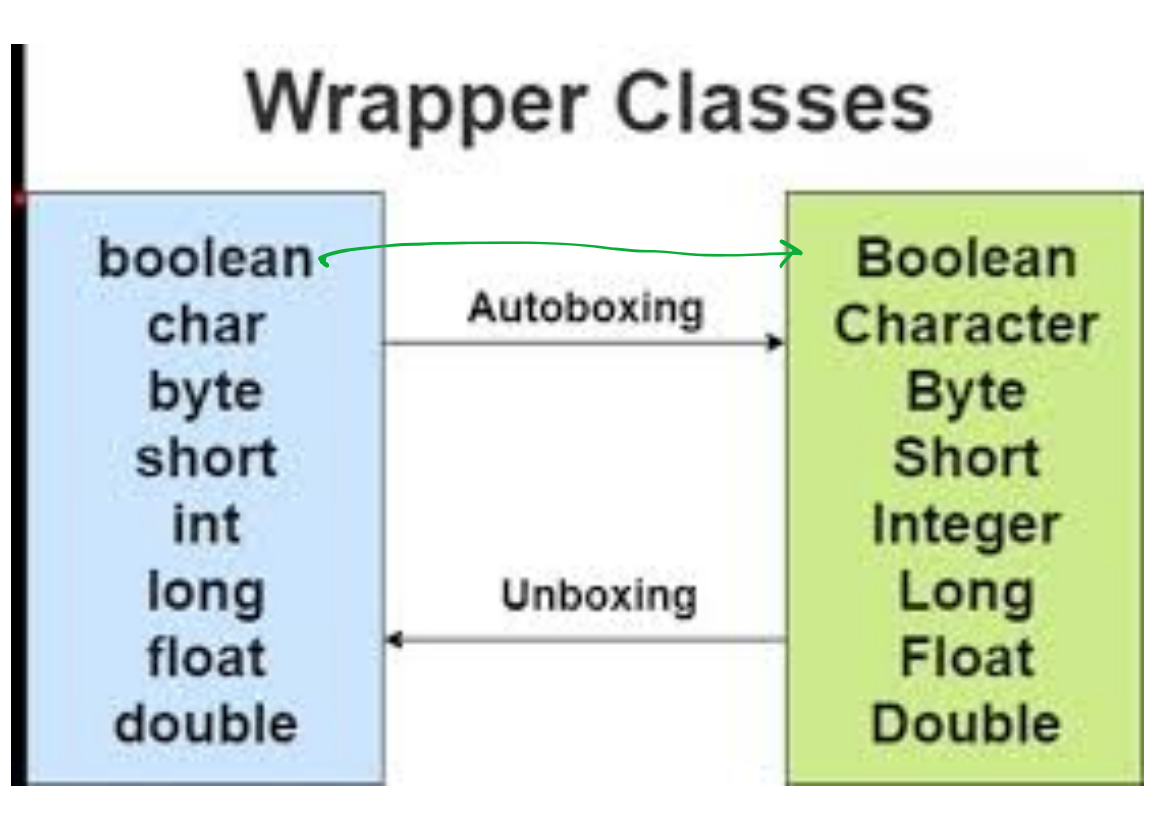
- byte
- short
- long
- int
- float
- double
- char
- boolean

non primitive



Primitive type	Wrapper Class
boolean	Boolean
byte	Byte
char	Character
float	Float
int	Integer
long	Long
short	Short
double	Double

String → Class



Wrapper Classes

boolean char byte short int long float double	Autoboxing → ← Unboxing	Boolean Character Byte Short Integer Long Float Double

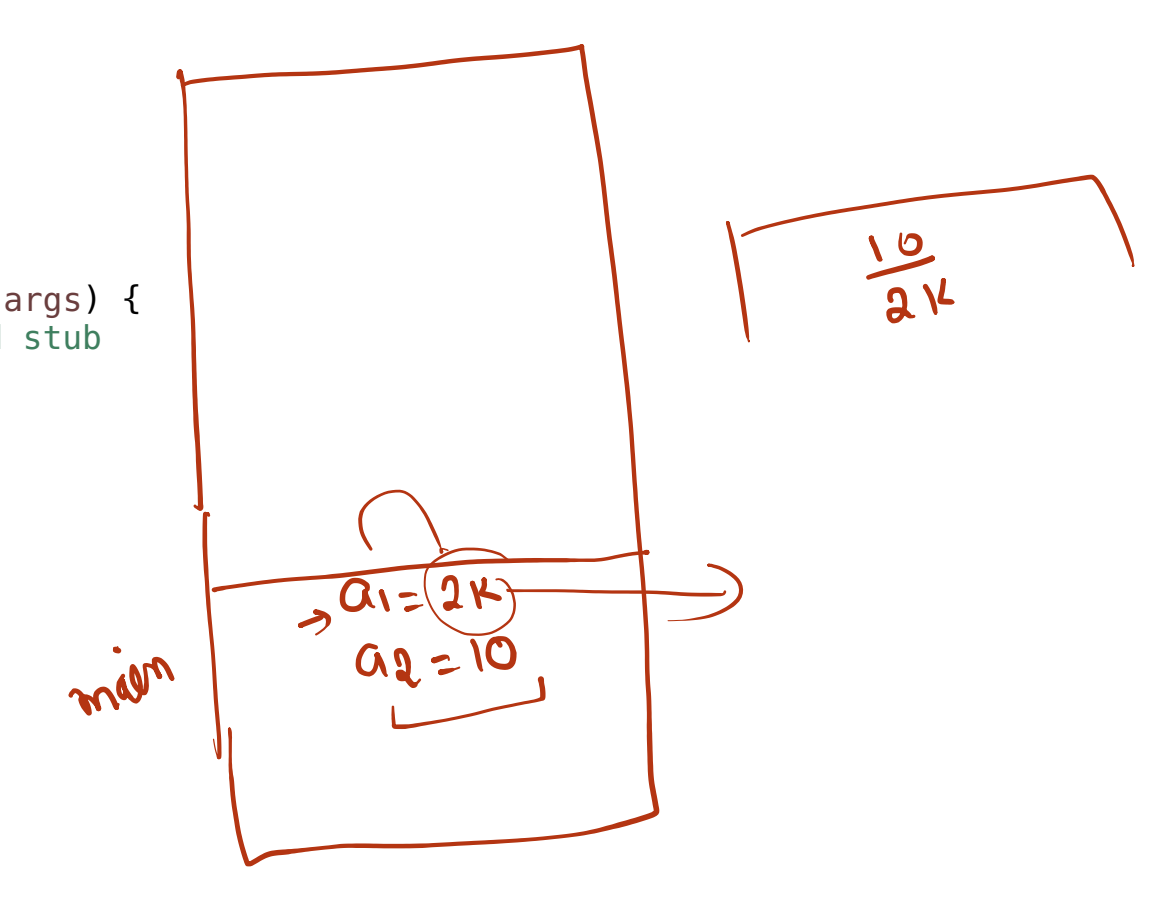
Stack → Heap

String s = "hello"

↳ toString()

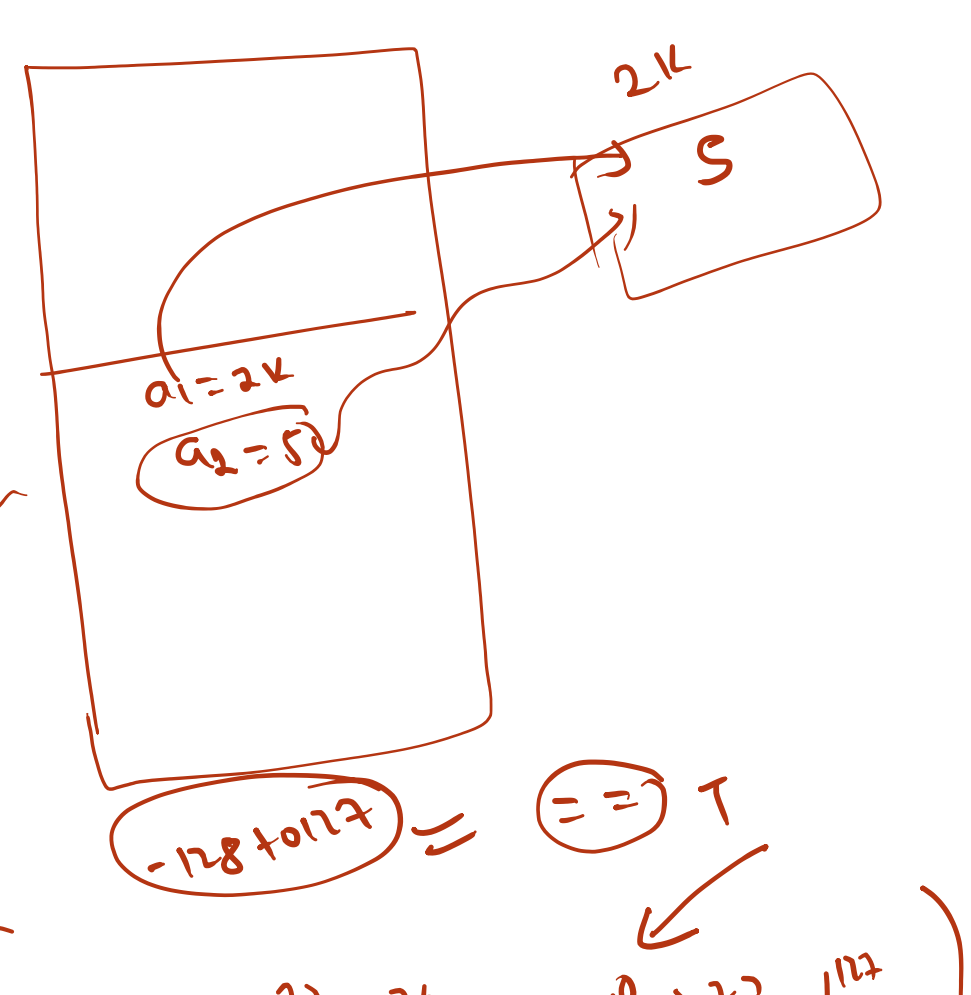
String ↳ content

```java
public static void main(String[] args) {
        // TODO Auto-generated method stub
        Integer a1 = 10;
        int a2 = 10;
        System.out.println(a1);
        System.out.println(a2);

    }
```



main
a1 = 2K
a2 = 10
10
2K

```java
public static void main(String[] args) {
        // TODO Auto-generated method stub
        Integer a1 = 10;
        int a2 = 5;
        System.out.println(a1);
        System.out.println(a2);
        Character ch = 'a';
        Long l = 899L;
        a1 = a2;// Auto-Boxing
        System.out.println(a1);
        a2 = a1;// Un-Boxing
        System.out.println(a2);

    }
```

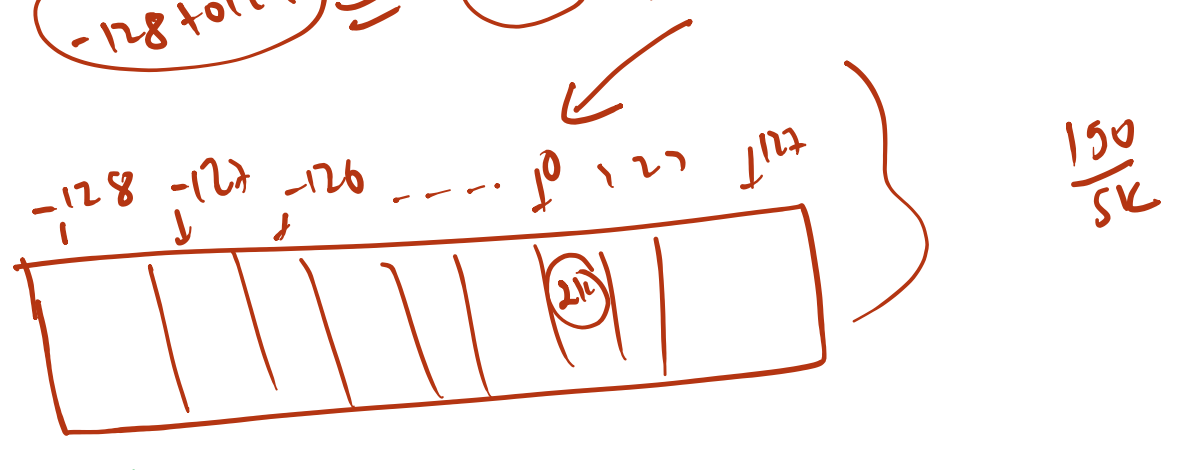


main
a1 = 2K
a2 = 5
2K
5

```java
Integer b1 = 19;
Integer b2 = 19;
Integer b3 = 190;
Integer b4 = 190;

System.out.println(b1 == b2);
System.out.println(b3 == b4);
```

5K
6K
T

-128 to 127 = == T



-128 -127 -126 ..... 0 1 2 127
2K
150
5K

cache → 2 5 6

low = -128;

high = h; +127

float ✗ double

Integer Long Byte char

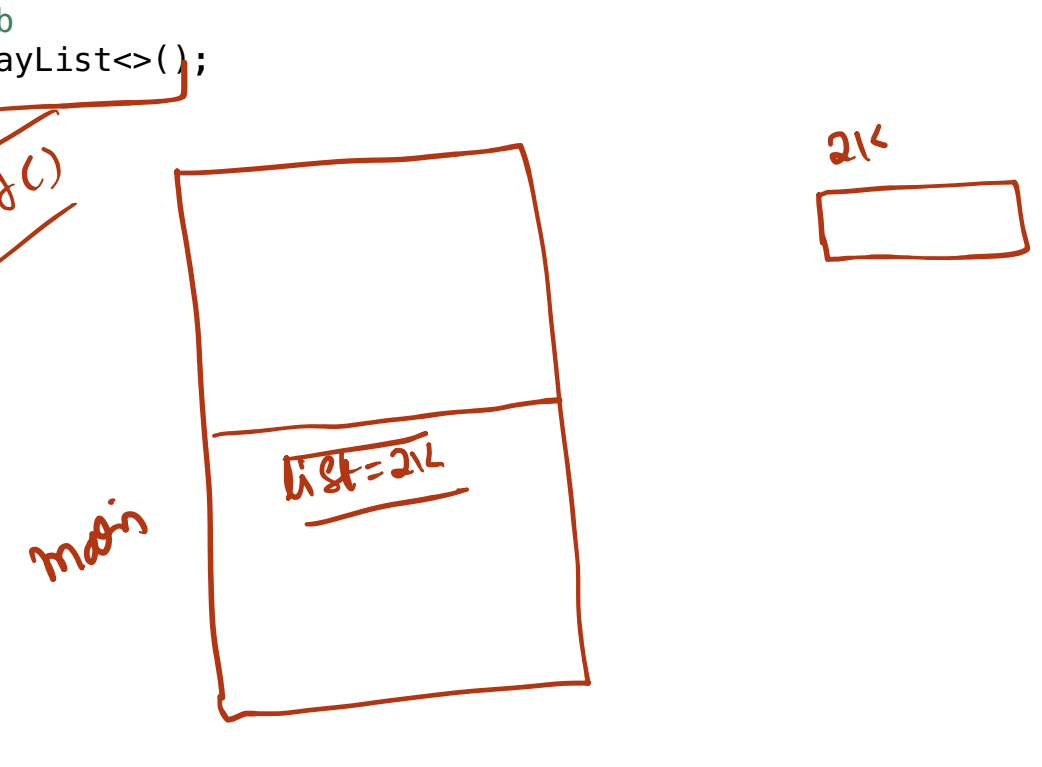
ArrayList ↳ class ↳ non primitive ↳ heap

& create

```java
public static void main(String[] args) {
        // TODO Auto-generated method stub
        ArrayList<Integer> list = new ArrayList<>();
    }
```

size

capacity()



main
list = 2K
2K

- add ✓
- get ✓
- remove
- update
- sort

"JIS RAASTE PE HUM CHAL RAHE HAI WOH HAI ONE WAY, NA REVERSE GEAR HAI NA U-TURN"

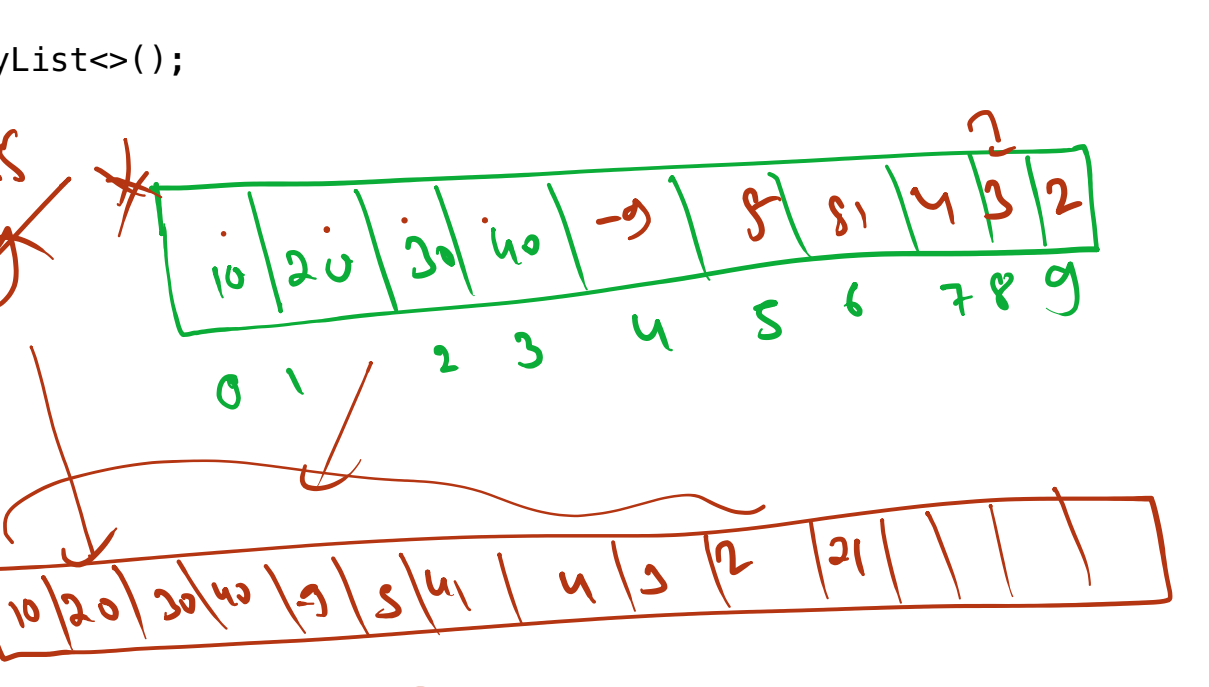
For →

for each ✓

value ✗

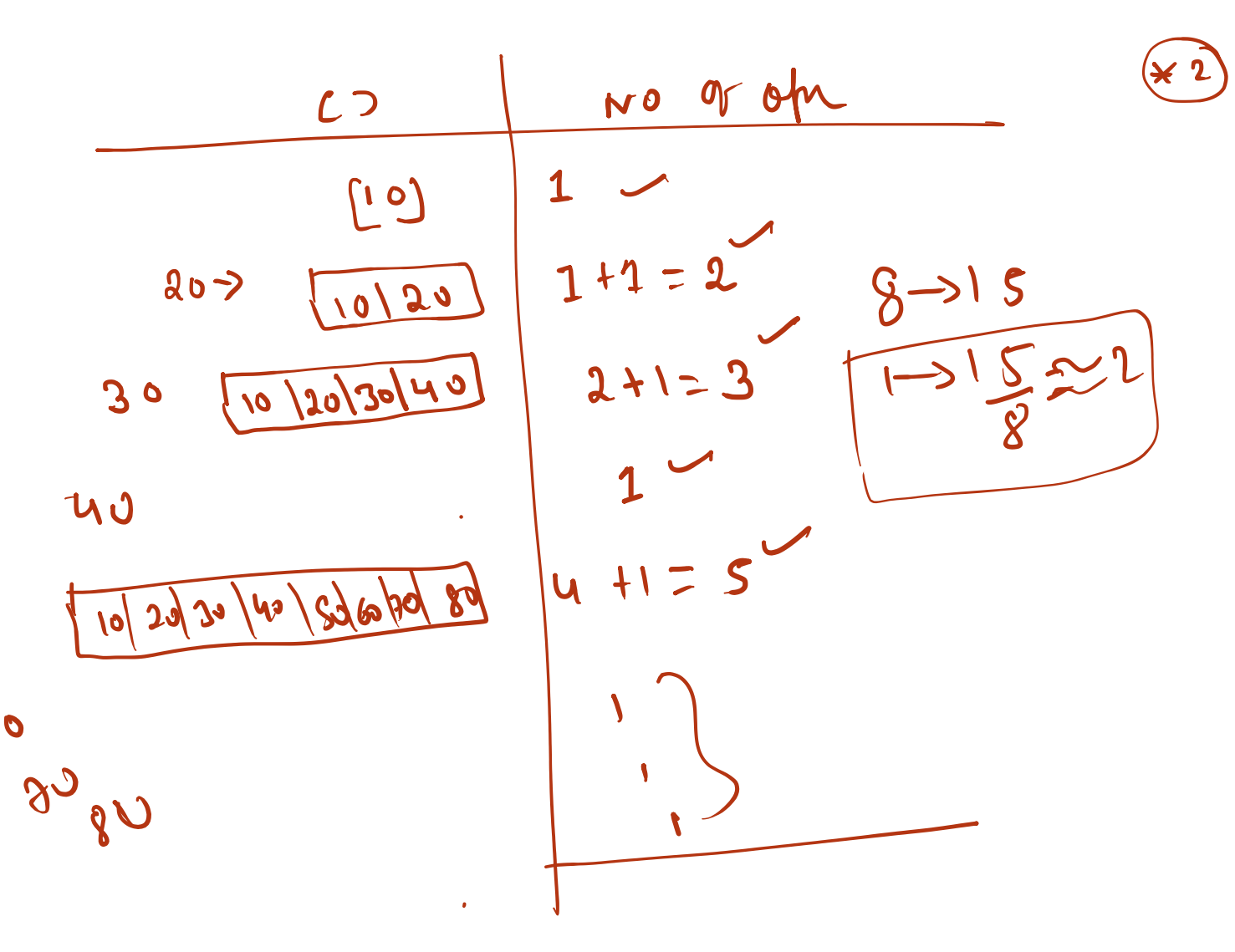
```java
public static void main(String[] args) {
        // TODO Auto-generated method stub
        ArrayList<Integer> list = new ArrayList<>();
        list.add(10);// O(1)
        list.add(20);
        list.add(30);
        list.add(40);
        list.add(, -9);// O(N)
        list.add(5);
        list.add(51);
        list.add(4);
        list.add(3);
        list.add(2);
        System.out.println(list.size());

    }
```

(oldcap)*1.5

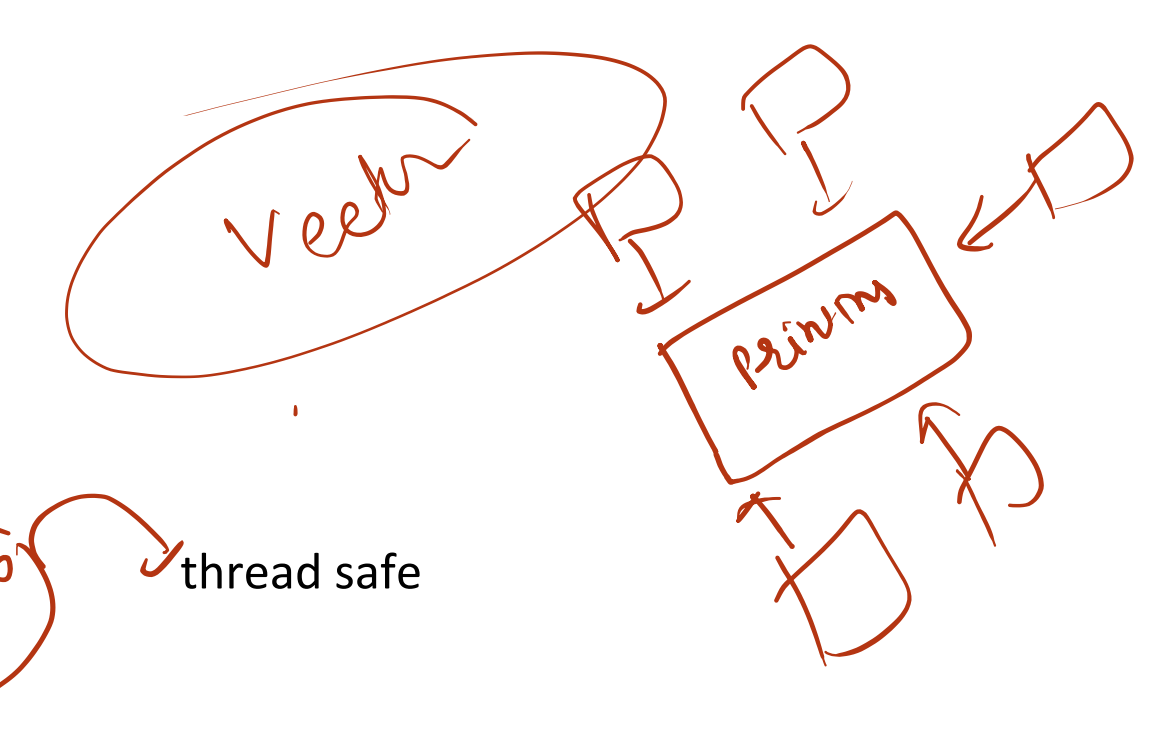


10 20 30 40 -9 5 51 4 3 2
0 1 2 3 4 5 6 7 8 9
10 20 30 40 -9 5 51 4 3 2 21

15×1.5 = 23

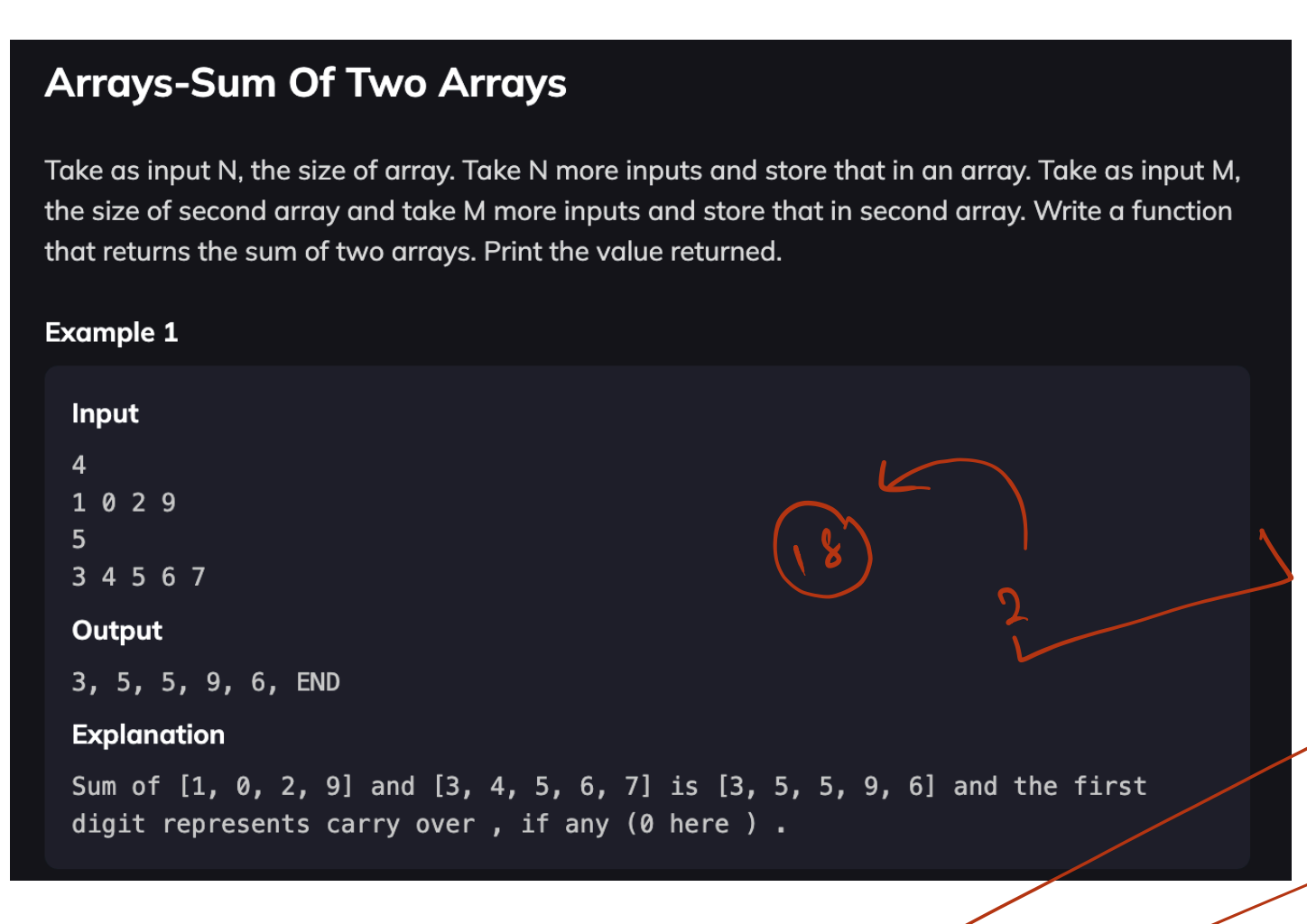


[10]
20 → [10|20]
30 [10|20|30|40]
40
50 [10|20|30|40|50|60|70|80]
60
70
80
No of opn
1 ✓
1+1 = 2 ✓
2+1 = 3 ✓
1 ✓
4+1 = 5 ✓
8 → 15
1 → 15/8 ≈ 2
×2



Vector
P
P
P
Prinvm
P
P

Vector thread safe



Arrays-Sum Of Two Arrays

Take as input N, the size of array. Take N more inputs and store that in an array. Take as input M, the size of second array and take M more inputs and store that in second array. Write a function that returns the sum of two arrays. Print the value returned.

Example 1

Input

```
4
1 0 2 9
5
3 4 5 6 7
```

Output

```
3, 5, 5, 9, 6, END
```

Explanation

```
Sum of [1, 0, 2, 9] and [3, 4, 5, 6, 7] is [3, 5, 5, 9, 6] and the first
digit represents carry over , if any (0 here ) .
```

18

2

2|2|2|

1|0|2|9

3|4|5|6|7

1 0 2 9
3 4 5 6 7
3 3 5 9 6

22
33
55

78
95
173

1 0 2 9
3 5 6 7 4
3 6 7 0 3

n
m ✓
m+1

16%10 = 6
7%10 = 7
7/10 = 0

carry = 0 1 0

[1, 0, 2, 9] [2, 4, 3, 5, 4, 7]

6|7|5|4

1 0 2 9
2 4 3 5 4 7

5 7 6

while (i >= 0 && j >= 0
Sum = carry + arr1[i] + arr2[j]
list.add(sum%10)
carry = sum/10
i--, j--

3

2|5|2|5|3 Reverse

```java
public static void Sum_Of_Two_Array(int[] arr1, int[] arr2) {
        int i = arr1.length - 1;
        int j = arr2.length - 1;
        int carry = 0;
        ArrayList<Integer> ll = new ArrayList<>();
        while (i >= 0 && j >= 0) {
            int sum = carry + arr1[i] + arr2[j];
            ll.add(sum % 10)
            carry = sum / 10;
            i--;
            j--;
        }

    }
```

3|2|4

3|4|9|2|8

i<0

3 2 4
3 4 9 2 8
3 5 2 5 2

```java
public static void Sum_Of_Two_Array(int[] arr1, int[] arr2) {
        int i = arr1.length - 1;
        int j = arr2.length - 1;
        int carry = 0;
        ArrayList<Integer> ll = new ArrayList<>();
        while (i >= 0 && j >= 0) {
            int sum = carry + arr1[i] + arr2[j];
            ll.add(sum % 10);
            carry = sum / 10;
            i--;
            j--;
        }
        while (i >= 0) {
            int sum = carry + arr1[i];
            ll.add(sum % 10);
            carry = sum / 10;
            i--;
        }
        while (j >= 0) {
            int sum = carry + arr2[j];
            ll.add(sum % 10);
            carry = sum / 10;
            j--;
        }

    }
```

1

2 3
9 8
2 1

1|2|1

C = 1